2026-8-6 Linux信号--信号保存

- 1. 信号保存的基本模型
  - 1.1 为什么需要保存信号
    - 1.1.1 产生与处理之间存在时间窗口
    - 1.1.2 信号从产生到递达之间的状态叫未决
    - 1.1.3 阻塞与忽略不是同一件事
- 2. 信号状态的数据表示
  - 2.1 block、pending 与 handler
    - 2.1.1 位图怎样表示信号集合
    - 2.1.2 标准信号不排队
  - 2.2 可递达状态与处理动作
    - 2.2.1 可递达状态
    - 2.2.2 handler 决定递达后做什么
    - 2.2.3 signal() 修改的是处理动作
  - 2.3 用 sigset_t 表示信号集合
    - 2.3.1 sigset_t 是操作系统提供的集合类型
    - 2.3.2 初始化、添加、删除和判断成员
    - 2.3.3 构造并打印信号集合
- 3. 用 sigprocmask 操作信号屏蔽字
  - 3.1 sigprocmask 的接口
  - 3.2 三种 how 的位运算语义
  - 3.3 保存旧屏蔽字并恢复
  - 3.4 阻塞只影响递达，不取消信号
  - 3.5 有两个信号不能被阻塞
- 4. 未决信号的查询与状态转换
  - 4.1 sigpending 的语义
  - 4.2 阻塞 SIGINT 后检查 pending 位
  - 4.3 查询的是状态窗口
  - 4.4 从阻塞保存到递达的状态转换与实验
    - 4.4.1 产生后先检查阻塞
    - 4.4.2 解除阻塞会使未决信号进入递达
    - 4.4.3 pending 位在进入 handler 前已经清零
    - 4.4.4 构造全信号集合
    - 4.4.5 屏蔽全部信号不等于进程无法被终止
- 5. 信号保存的逻辑闭环
  - 5.1 一次信号保存在内核中的完整变化

- 5.2 三组关系的结论

1. 信号保存的基本模型

1.1 为什么需要保存信号

1.1.1 产生与处理之间存在时间窗口

信号产生以后不一定能立即处理，因为进程可能正在执行关键内核逻辑，或者暂时不允许处理某个信号。从信号产生到真正执行处理动作之间存在一个时间窗口，内核必须把这个尚未处理的信号保存下来。

```
信号产生
    |
    v
信号保存在进程的待处理集合中
    |
    v
满足递达条件
    |
    v
执行默认动作、忽略动作或自定义处理函数
```

因此，“产生”和“递达”不是同义词：

- **产生**：事件已经发生，内核已经决定向哪个进程写信号。
- **保存**：信号已经记录在进程控制信息中，还没有执行处理动作。
- **递达**：信号正式开始执行处理动作。

1.1.2 信号从产生到递达之间的状态叫未决

信号从产生到递达之间的状态称为 **Pending（未决）**。未决信号还没有执行默认动作、忽略动作或用户处理函数，只是暂时被内核保存。
进程还可以选择 **Block（阻塞）** 某个信号：

- 信号没有被阻塞时，产生后可以进入递达流程。
- 信号被阻塞时，即使已经产生，也会保持在未决状态。
- 解除阻塞以后，未决信号才有机会递达。

1.1.3 阻塞与忽略不是同一件事

概念	所处阶段	本质
阻塞 Block	递达之前	阻止信号进入处理流程
忽略 Ignore	递达以后	已经处理，但处理动作是什么也不做
未决 Pending	产生以后、递达以前	信号被保存的状态

阻塞不是一种“处理方式”，而是延后递达的条件；忽略是实际被选中的处理动作。

进程即使把某个信号设置为 `SIG_IGN`，该信号也不是永远处于未决状态。只要没有被阻塞，信号仍会递达，只是递达后的动作是忽略。

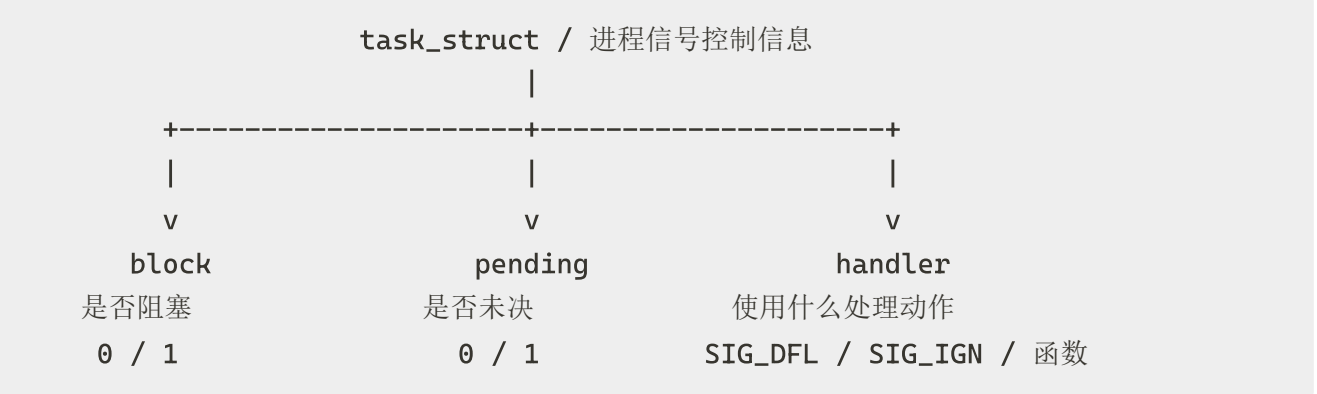
2. 信号状态的数据表示

2.1 block、pending 与 handler

可以把内核为进程维护的信号状态抽象成三张表：

表	作用	每个信号保存的信息
`block`	信号屏蔽字	该信号是否被阻塞
`pending`	未决信号集合	该信号是否已经产生但尚未递达
`handler`	信号处理动作表	默认动作、忽略或自定义处理函数

概念关系可以表示为：



现代 Linux 内核的实际数据结构更复杂，`block`、`pending` 和处理动作可能由不同的内核结构共同维护。这里使用“三张表”作为理解模型，不把它等同于某一个具体字段的物理布局。

2.1.1 位图怎样表示信号集合

普通信号可以用位图描述：

- 位的位置对应信号编号。
- 位的值为 1，表示该集合包含这个信号。
- 位的值为 0，表示该集合不包含这个信号。
  例如，把 `pending` 中编号为 2 的位设为 1，表示 `SIGINT` 已经产生但尚未递达：

```
pending 中 SIGINT 位 = 1  ->  SIGINT 未决
block   中 SIGINT 位 = 1  ->  SIGINT 被阻塞
```

位图保存的是状态，不是完整事件内容。普通信号主要记录“哪个编号处于未决状态”，不会把每次产生都扩展成一条独立记录。

2.1.2 标准信号不排队

对普通标准信号而言，`pending` 中同一个信号只有一个状态位。某个标准信号阻塞期间若重复产生多次，位仍然只是 1，解除阻塞后通常只递达一次。这说明重复的标准信号可能合并，不能把 `pending` 位图理解为“信号队列长度”。
实时信号通常具有排队语义，多个相同实时信号可以分别记录；实时信号的完整排队规则不属于本笔记主线。

2.2 可递达状态与处理动作

2.2.1 可递达状态

一个信号是否能够递达，至少需要同时满足：

1. 该信号已经产生，`pending` 对应位为 1。
2. 该信号没有被阻塞，`block` 对应位为 0。
   可以用位运算表示：

```
可递达信号 = pending & ~block
```

例如只考虑编号 2 的 `SIGINT`：

pending	block	pending & ~block	结果
1	1	0	已产生，但仍未决
1	0	1	可以递达
0	1	0	未产生，阻塞位本身不会制造信号
0	0	0	未产生，也没有可递达信号

阻塞位为 1 只会阻止已经产生的信号递达，它不会凭空产生信号。

2.2.2 handler 决定递达后做什么

信号具备递达条件以后，内核还要根据 `handler` 表决定处理动作：

```
可以递达的信号
        |
        v
   handler 表中的动作
        |
        +-- SIG_DFL：执行系统默认动作
        +-- SIG_IGN：忽略，什么也不做
        +-- 用户函数：执行自定义 handler
```

所以：

- 阻塞决定“现在能不能递达”。
- handler 决定“递达以后做什么”。
- pending 记录“是否已经产生但尚未递达”。

2.2.3 signal() 修改的是处理动作

例如：

```
signal(SIGINT, myhandler);
```

主要修改的是 `handler` 表中 `SIGINT` 对应的处理动作，不会主动产生 `SIGINT`，也不会自动修改该信号的阻塞位。

`SIG_DFL` 与 `SIG_IGN` 可以理解为特殊的处理动作：

```
signal(SIGINT, SIG_DFL);  // 使用默认动作
signal(SIGINT, SIG_IGN);  // 忽略
```

它们与普通自定义处理函数一样，都是在修改“未来收到信号时怎样处理”的配置。

从概念上看，`handler` 表可以理解为一个按信号编号索引的函数指针数组，每个表项保存 `SIG_DFL`、`SIG_IGN` 或用户函数地址。这张表在信号产生之前就已经存在，因此进程在信号到来时才有现成的处理动作可以查询。具体内核结构不一定是一个简单的 32 项数组，但“提前登记处理动作，收到信号后查表”这一模型是成立的。

`block`、`pending` 和 `handler` 都属于内核维护的数据，用户程序不能直接从一个进程改写另一个进程的这三张表，只能通过 `signal`、`sigprocmask`、`sigpending` 等接口请求内核访问或修改。

2.3 用 sigset_t 表示信号集合

2.3.1 sigset_t 是操作系统提供的集合类型

`sigset_t` 是操作系统提供的信号集合类型，用来表示一组信号。用户程序不应自行假定它只有一个整数，也不应直接操作它的内部二进制布局，而应使用标准接口完成构造、修改和成员判断。

```
#include <signal.h>

sigset_t set;
```

同一个 `sigset_t` 类型可以承载不同语义的集合：

- 作为 `sigprocmask()` 的输入，表示要阻塞、解除阻塞或设置的信号。
- 作为 `sigpending()` 的输出，接收当前未决信号集合。

- 作为临时集合，用于逐项检查信号编号。

2.3.2 初始化、添加、删除和判断成员

常用接口如下：

```
#include <signal.h>

int sigemptyset(sigset_t *set);
int sigfillset(sigset_t *set);
int sigaddset(sigset_t *set, int signum);
int sigdelset(sigset_t *set, int signum);
int sigismember(const sigset_t *set, int signum);
```

接口	作用
`sigemptyset`	清空集合，所有成员位为 0
`sigfillset`	把集合填满，包含可加入的全部信号
`sigaddset`	加入一个信号
`sigdelset`	删除一个信号
`sigismember`	判断一个信号是否在集合中

返回值：

- 前四个函数成功返回 0，失败返回 -1，并设置 `errno`。
- `sigismember()` 返回 1 表示是成员，返回 0 表示不是成员，返回 -1 表示出错。

2.3.3 构造并打印信号集合

下面的函数按照 31 到 1 的顺序打印集合成员：

```
#include <iostream>
#include <signal.h>

void PrintPending(const sigset_t &set)
{
    for (int signo = 31; signo >= 1; --signo)
    {
```

```
        std::cout << (sigismember(&set, signo) == 1 ? '1' : '0');

        if ((signo - 1) % 8 == 0 && signo != 1)
            std::cout << ' ';
    }
    std::cout << std::endl;
}
```

输出中的每一位对应一个信号编号。应使用 `sigismember()` 读取状态，不能把 `sigset_t` 直接当成普通整数打印。

3. 用 sigprocmask 操作信号屏蔽字

3.1 sigprocmask 的接口

```
#include <signal.h>

int sigprocmask(int how, const sigset_t *set, sigset_t *oldset);
```

参数含义：

- `how`：指定如何修改当前信号屏蔽字。
- `set`：希望参与修改的信号集合。
- `oldset`：若不为空，用来接收修改前的信号屏蔽字。
- `set` 为空时，可以用来查询当前信号屏蔽字。

成功返回 0，失败返回 -1，并设置 `errno`。

3.2 三种 how 的位运算语义

how	含义	概念运算
SIG_BLOCK	增加要阻塞的信号	mask = mask | set
SIG_UNBLOCK	解除集合中的阻塞	mask = mask & ~set
SIG_SETMASK	直接替换整个屏蔽字	mask = set

因此：

- 阻塞只是把对应位置 1。
- 解除阻塞只是把对应位清 0。
- 设置屏蔽字会替换原来的整个集合，而不是只修改某个成员。

3.3 保存旧屏蔽字并恢复

修改屏蔽字前通常要保存旧值：

```
#include <signal.h>

sigset_t block_set;
sigset_t old_set;

sigemptyset(&block_set);
sigaddset(&block_set, SIGINT);

sigprocmask(SIG_BLOCK, &block_set, &old_set);

// 临时屏蔽 SIGINT

sigprocmask(SIG_SETMASK, &old_set, nullptr);
```

恢复旧集合很重要：直接构造一个空集合并调用 `SIG_SETMASK`，会覆盖调用者原来的全部屏蔽状态，而不是只恢复本次添加的 `SIGINT`。

3.4 阻塞只影响递达，不取消信号

对 `SIGINT` 设置阻塞以后：

- 键盘继续产生 `SIGINT` 时，`pending` 中对应位可以变为 1。
- 因为 `block` 中对应位也是 1，信号暂时不会递达。
- 解除阻塞后，待处理信号进入递达流程。

只要对应位已经从 1 变为 0，并且没有其他屏蔽条件，信号就会开始执行处理动作。处理动作本身如何运行，是下一阶段的问题。

3.5 有两个信号不能被阻塞

`SIGKILL`（9）和 `SIGSTOP`（19）既不能被捕获或忽略，也不能被阻塞：

- `SIGKILL` 始终可以终止进程。
- `SIGSTOP` 始终可以暂停进程。

因此，试图用屏蔽字“屏蔽所有信号”也不能阻止这两个强制控制信号。

4. 未决信号的查询与状态转换

4.1 sigpending 的语义

```
#include <signal.h>

int sigpending(sigset_t *set);
```

`sigpending()` 把当前线程可观察到的未决信号写入 `set`：

- 成功返回 0。
- 失败返回 -1，并设置 `errno`。

它会报告已经产生但目前尚未递达的信号，包括被屏蔽而停留的未决信号。查询未决集合不会修改信号状态，也不会解除阻塞。

4.2 阻塞 SIGINT 后检查 pending 位

下面的程序先阻塞 `SIGINT`，再给自己发送 `SIGINT`，随后持续查询并打印 pending：

```
#include <iostream>
#include <signal.h>
#include <unistd.h>

void PrintPending(const sigset_t &set)
{
    for (int signo = 31; signo >= 1; --signo)
        std::cout << (sigismember(&set, signo) == 1 ? '1' : '0');
    std::cout << std::endl;
}

int main()
{
    sigset_t block_set;
    sigset_t old_set;
```

```
    sigset_t pending;

    sigemptyset(&block_set);
    sigaddset(&block_set, SIGINT);

    if (sigprocmask(SIG_BLOCK, &block_set, &old_set) == -1)
        return 1;

    raise(SIGINT);

    while (true)
    {
        sigemptyset(&pending);

        if (sigpending(&pending) == 0)
            PrintPending(pending);

        sleep(1);
    }
}
```

预期看到的重点不是其余位，而是 `SIGINT` 对应的未决位为 1。因为 `SIGINT` 同时被阻塞，所以它不会立刻进入默认终止流程。

4.3 查询的是状态窗口

`pending` 位在信号递达前会被清 0。因此，能否在 `sigpending()` 中看到某一位为 1，取决于查询发生在“信号已经产生”之后、“处理动作开始”之前。

如果信号没有被阻塞并很快递达，这个窗口可能极短；主动阻塞信号，可以稳定地观察未决状态。

4.4 从阻塞保存到递达的状态转换与实验

4.4.1 产生后先检查阻塞

```
信号产生
   |
   v
设置 pending 对应位
   |
```

```
   v
该信号是否被阻塞？
   |
   +-- 是：保持 Pending，等待解除阻塞
   |
   +-- 否：进入递达流程
```

这条路径说明了为什么“发送信号”不一定会立即调用处理函数：

- 阻塞决定信号是否暂时停在未决状态。
- pending 记录它确实已经到达。
- 解除阻塞后才能继续向处理阶段前进。

4.4.2 解除阻塞会使未决信号进入递达

假设 `SIGINT` 的 `pending=1`、`block=1`：

```
产生 SIGINT         pending: 0 -> 1
保持阻塞            block:   1
解除对 SIGINT 的阻塞
block: 1 -> 0
pending & ~block 变为 1
```

此后，内核在进程进入合适的返回点时处理这个可递达信号。若采用默认动作，进程可能立即终止；若采用自定义处理动作，内核会在进入用户态处理函数前清除相应 pending 位。

4.4.3 pending 位在进入 handler 前已经清零

为了验证信号位图什么时候从 1 变为 0，可以在信号处理函数内部再次调用 `sigpending()`。

```cpp
#include <iostream>
#include <signal.h>
#include <unistd.h>

void PrintPending(const sigset_t &set)
{
    for (int signo = 31; signo >= 1; --signo)
        std::cout << (sigismember(&set, signo) == 1 ? '1' : '0');
    std::cout << std::endl;
}
```

```cpp
void handler(int signo)
{
    sigset_t pending;
    sigemptyset(&pending);
    sigpending(&pending);

    std::cout << "enter handler: " << signo << std::endl;
    PrintPending(pending);
    std::cout << "leave handler" << std::endl;
}

int main()
{
    signal(SIGINT, handler);

    sigset_t block_set;
    sigset_t old_set;
    sigemptyset(&block_set);
    sigaddset(&block_set, SIGINT);
    sigprocmask(SIG_BLOCK, &block_set, &old_set);

    raise(SIGINT);

    sigset_t pending;
    sigemptyset(&pending);
    sigpending(&pending);
    PrintPending(pending);

    sigprocmask(SIG_SETMASK, &old_set, nullptr);
    pause();
}
```

在解除阻塞之前，`SIGINT` 的 pending 位为 1；进入 handler 后重新查询，该位已经为 0。这说明内核是在调用自定义处理函数之前清除 pending 位，而不是等 handler 返回以后才清零。

```
0000000 00000000 00000000 00000010
0000000 00000000 00000000 00000010
解除对 2 号信号的屏蔽，待处理的信号将被递达
======================entery handler============
0000000 00000000 00000000 00000000
处理完成: 2
=====================leave handler============
0000000 00000000 00000000 00000000
0000000 00000000 00000000 00000000
```

4.4.4 构造全信号集合

可以清空集合后逐个加入 1-31 号信号，也可以使用 `sigfillset()` 构造完整集合。随后通过 `sigprocmask(SIG_SETMASK, ...)` 将其设置为屏蔽字：

```cpp
#include <iostream>
#include <signal.h>
#include <unistd.h>

void PrintPending(const sigset_t &set)
{
    for (int signo = 31; signo >= 1; --signo)
        std::cout << (sigismember(&set, signo) == 1 ? '1' : '0');
    std::cout << std::endl;
}

int main()
{
    sigset_t block_set;
    sigset_t old_set;

    sigemptyset(&block_set);

    for (int signo = 1; signo <= 31; ++signo)
        sigaddset(&block_set, signo);

    sigprocmask(SIG_SETMASK, &block_set, &old_set);

    while (true)
    {
        sigset_t pending;
        sigemptyset(&pending);
```

```
        sigpending(&pending);
        PrintPending(pending);
        sleep(1);
    }
}
```

```
ZL@VM-0-14-ubuntu:~/code/lesson28$ kill -1 3170466
ZL@VM-0-14-ubuntu:~/code/lesson28$ kill -2 3170466
ZL@VM-0-14-ubuntu:~/code/lesson28$ kill -3 3170466
ZL@VM-0-14-ubuntu:~/code/lesson28$ kill -4 3170466
ZL@VM-0-14-ubuntu:~/code/lesson28$ kill -5 3170466
ZL@VM-0-14-ubuntu:~/code/lesson28$ kill -6 3170466
ZL@VM-0-14-ubuntu:~/code/lesson28$ kill -7 3170466
ZL@VM-0-14-ubuntu:~/code/lesson28$ kill -8 3170466
ZL@VM-0-14-ubuntu:~/code/lesson28$
```

```
00000000000000000000000001111111
00000000000000000000000001111111
00000000000000000000000001111111
00000000000000000000000001111111
00000000000000000000000001111111
00000000000000000000000001111111
00000000000000000000000011111111
00000000000000000000000011111111
00000000000000000000000011111111
00000000000000000000000011111111
```

4.4.5 屏蔽全部信号不等于进程无法被终止

即使程序尝试把 1-31 号信号都加入屏蔽集合，`SIGKILL` 和 `SIGSTOP` 仍然不能真正被阻塞：

- `kill -9 PID` 仍然可以终止进程。
- `kill -19 PID` 仍然可以暂停进程。

另外，屏蔽大量信号会让正常的退出、恢复和错误通知长期停留在 pending 状态，因此实际程序通常只阻塞必要的最小集合，并在有限时间内恢复原屏蔽字。

5. 信号保存的逻辑闭环

5.1 一次信号保存在内核中的完整变化

```
信号产生
   |
   v
pending[signum] = 1
   |
   v
block[signum] 是否为 1?
   |
   +-- 是：保持 Pending
   |         |
   |         +-- sigpending() 可以观察到该位
   |         +-- 阻塞期间标准信号通常合并为一次未决
   |
   +-- 否：可递达
             |
```

```
        v
pending[signum] 清 0
        |
        v
handler 决定默认、忽略或自定义动作
```

5.2 三组关系的结论

- **产生与保存**：产生说明内核已经写入信号；保存说明它尚未完成递达。
- **未决与阻塞**：未决描述“已经来但没处理”，阻塞描述“暂不允许递达”。
- **阻塞与忽略**：阻塞在递达前拦住信号，忽略是递达后的处理动作。
- **屏蔽字与待处理集合**：`sigprocmask` 操作 block，`sigpending` 查询 pending。
- **保存与处理**：pending 位在进入 handler 前清零，handler 执行不属于本次保存阶段。
- **普通信号与实时信号**：普通信号由位图保存且可能合并，实时信号通常具有排队语义。

因此，信号保存阶段的逻辑终点是：内核已经保存信号，并在阻塞解除后判断它可以递达；进入处理动作以后，才属于信号处理阶段。